

Software Design and Architecture

Lecture-2

By: Usama Musharaf



Requirement Engineering

What is Requirement?

What is requirement?

- The descriptions of what the system **should do**
- **services** that it provides and the **constraints**

Importance of Requirements

- **Reduces project failure**
- **Improves customer satisfaction**

Common Causes of Failure

- Incomplete requirements
- Misunderstood needs
- Changing requirements

Types of Requirements

1. Functional Requirements

how system reacts to input

how system behaves in particular situation

Examples:

- The system shall allow users to register.
- The system shall generate monthly sales reports.
- The ATM shall dispense cash after PIN verification.

Types of Requirements

2. Non-Functional Requirements

How the system should perform

Define quality attributes and constraints.

a) Performance

The system shall respond within 2 seconds.

The system shall handle 10,000 concurrent users.

b) Security

All passwords must be encrypted.

Only admins can delete records.

c) Usability

A new user should learn the system in under 30 minutes.

The UI must support English and Urdu.

d) Reliability

System uptime must be 99.9%.

Backup should run every 24 hours.

Types of Requirements

3. User Requirements

High-level needs in user language

Written for customers and end-users.

Examples:

- Users want to track their orders.
- Teachers need to upload assignments.
- Students want online results.

4. Domain Requirements

Rules from business or industry

Come from the problem domain.

Examples:

- In banking: Interest must be calculated daily.
- In aviation: System must follow FAA standards.

Types of Requirements

5. Interface Requirements

How system interacts with other systems/users

Examples:

- The system shall provide REST APIs.
- The system shall integrate with Google Maps.

6. Data Requirements

How data is stored and managed

Examples:

- Customer records must be stored for 5 years.
- Each user has a unique ID.
- Logs must be retained for auditing.

Types of Requirements

7. Constraint Requirements

Limitations on solution

Examples:

- System must be developed in Java.
- Must run on Linux servers.

8. Business Requirements

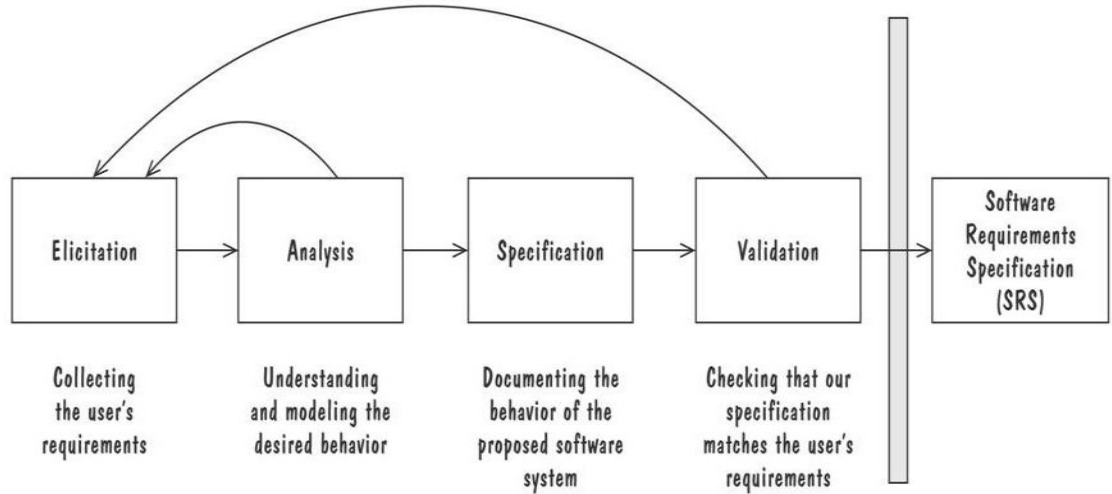
Organizational goals

Examples:

- Reduce customer service cost by 30%.
- Launch system within 6 months.
- Support 5 new branches.

Requirement Engineering Process

- Performed by the req. analyst or system analyst
- The final outcome is a Software Requirements Specification (SRS) document



Requirement Elicitation

Step-1: Identify Stakeholders

Identify source of requirements

- Interviewing stakeholders
- Reviewing available documentations
- Observing the current system (if one exists)

Requirement Elicitation

Step-1: Identify Stakeholders

STAKEHOLDERS

- Clients: pay for the software to be developed
- Users: use the system
- Domain experts: familiar with the problem that the software must automate

Stakeholder List

- Auditor
- Buyer
- Clerical user
- Customer service analyst
- Database administrator
- Financial expert
- Sales specialist
- Software Architect
- Network Administrator
- Usability specialist
- Security Specialist

Requirement Elicitation

Step-2: Understand the Problem Domain

Learn the business and environment.

Activities:

- Study existing system
- Read documents
- Observe workflows

Example:

- Before building a hospital system, understand patient flow.

Requirement Elicitation

Step-3: Select Elicitation Techniques

Decide how to gather requirements.

Common techniques:

- Interviews
- Questionnaires
- Workshops
- Observation
- Prototyping

Requirement Analysis

Analysis results in requirements models.

Requirements models (also referred to as analysis models) are user requirements represented by diagrams.

Requirement Analysis

Domain Model

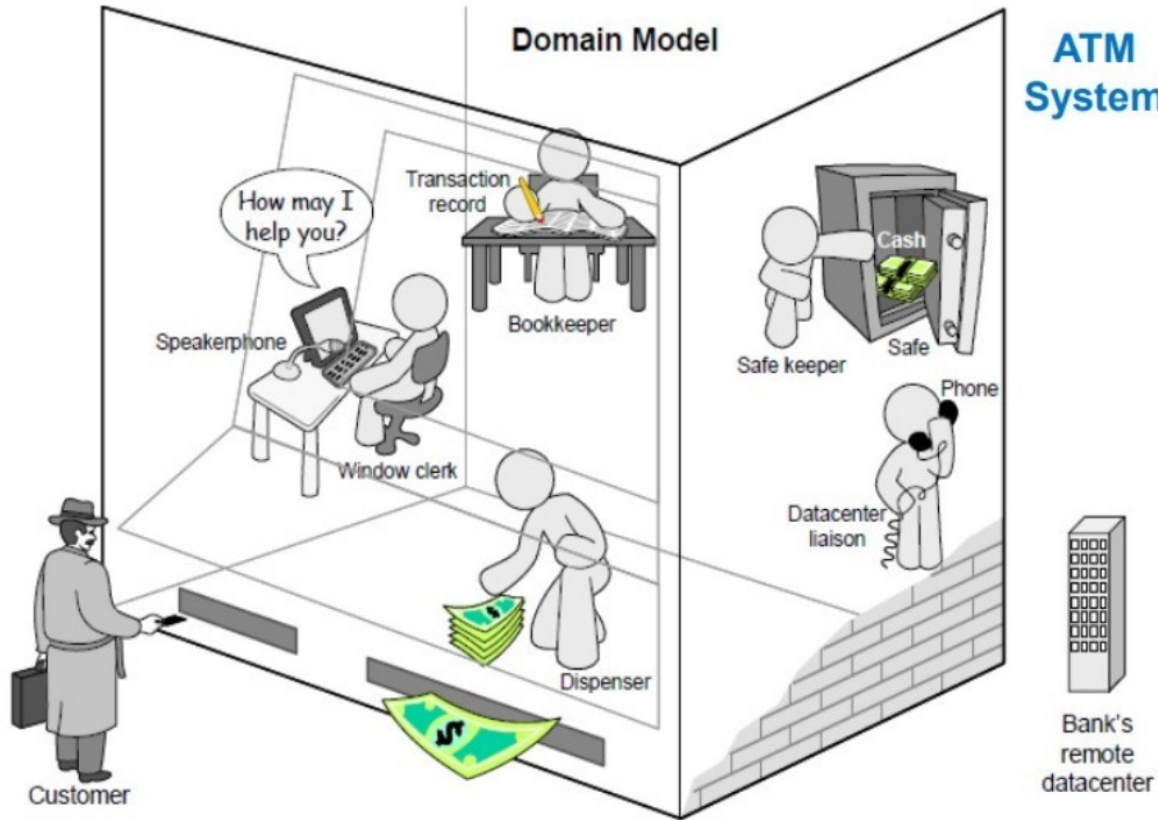
A Domain Model is a conceptual representation of the real-world problem domain showing:

- Important entities (concepts)
- Their attributes
- Their relationships

It describes what exists in the business world, not how the software is built.

Requirement Analysis

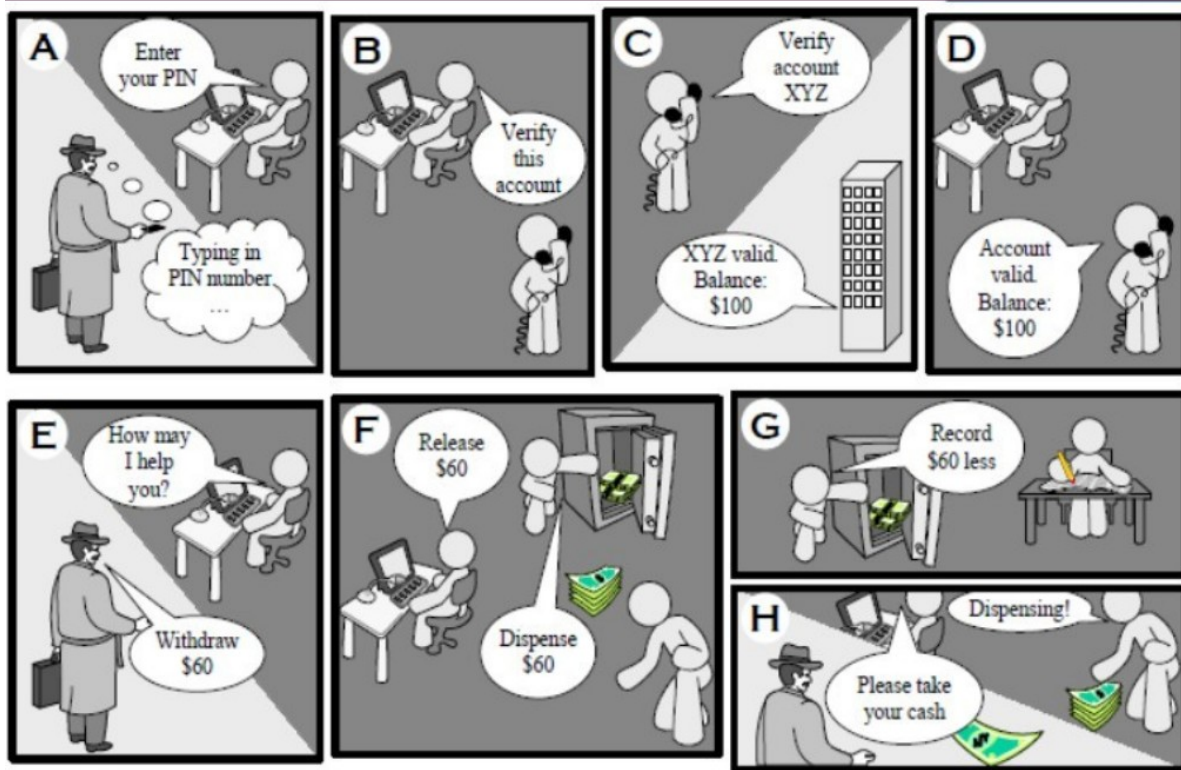
Domain Model



Imagined static structure of ATM shows internal components and their roles.

Requirement Analysis

Domain Model

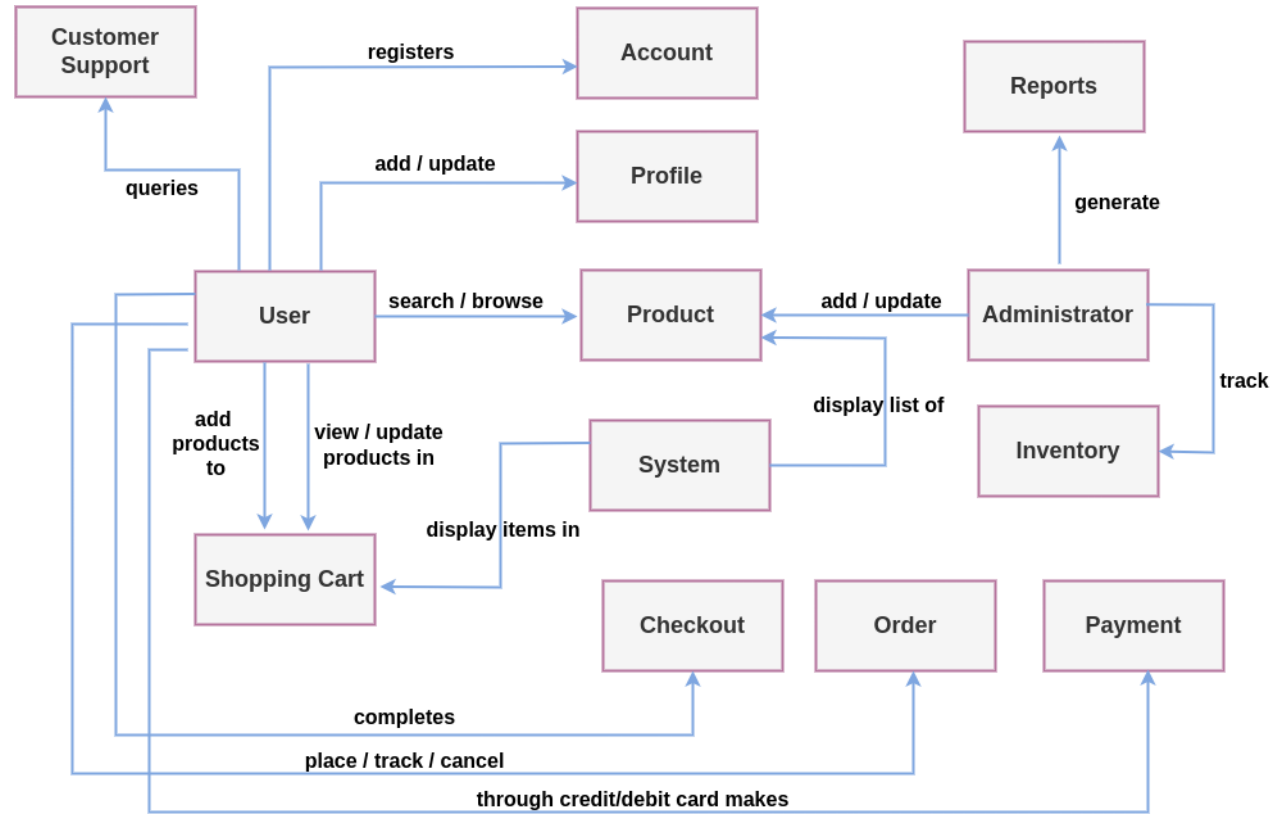


Dynamic interactions of the imagined components during task accomplishment.

ATM System

Requirement Analysis

Domain Model



Data Flow Diagram:

A data flow diagram (DFD) is a graphical tool that allows system analysts (and system users) to depict the flow of data in an information system.

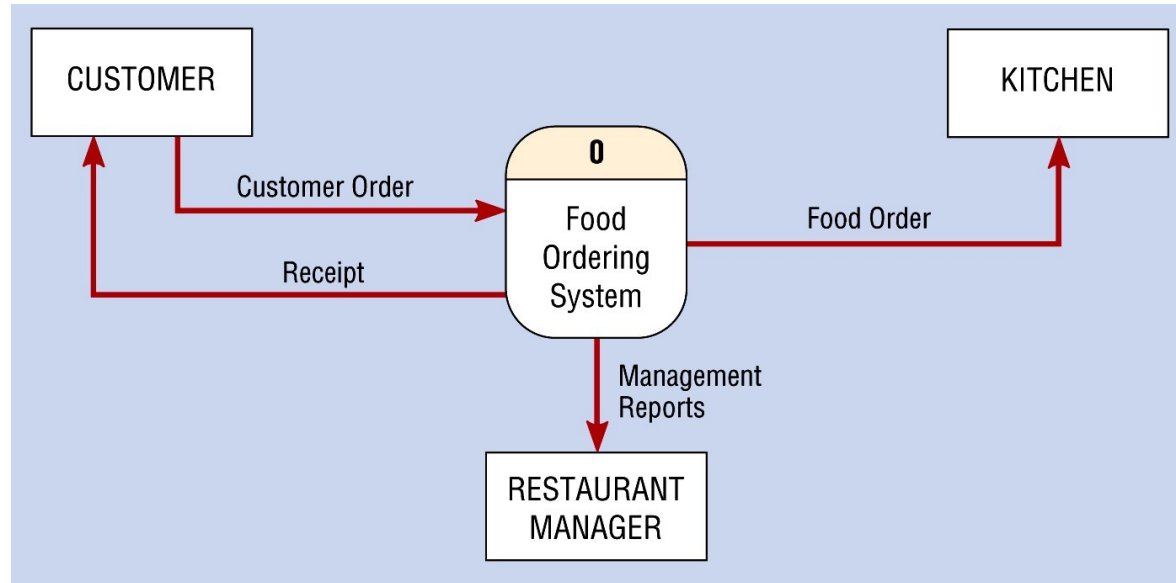
DFD Steps:

- Context Diagram
- Level-0 DFD
- Level-1 DFD and so on

Requirement Analysis

Data Flow Diagram

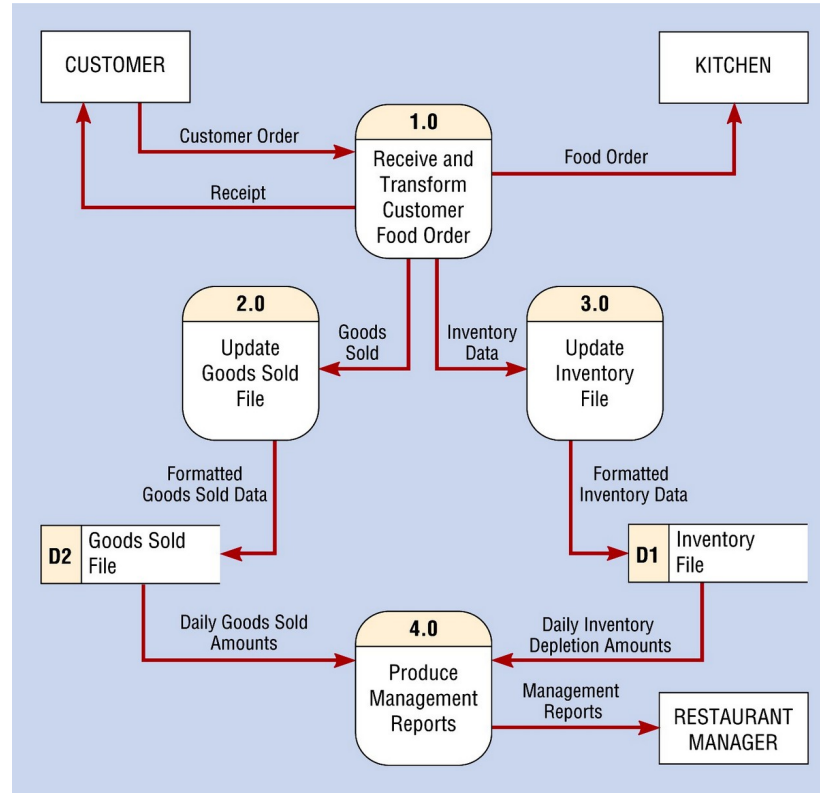
CONTEXT DIAGRAM OF FOOD ORDERING SYSTEM



Requirement Analysis

Data Flow Diagram

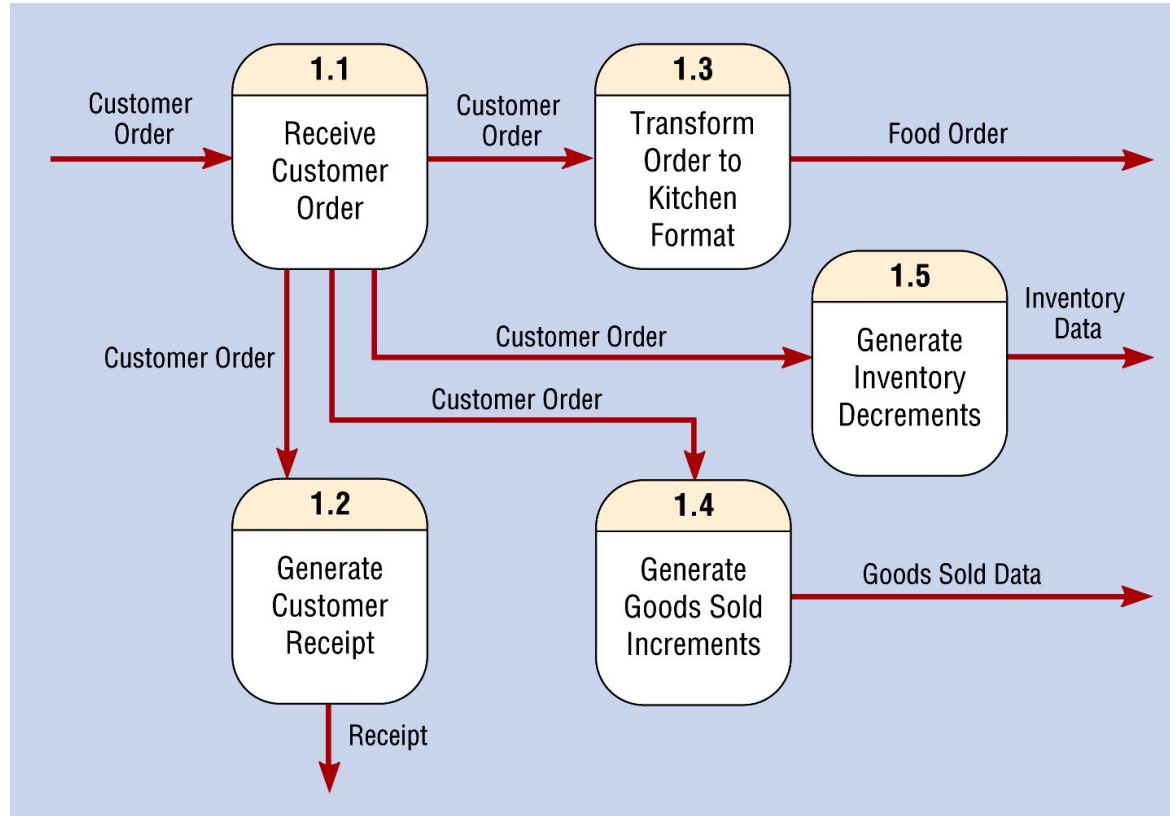
Level-0 DFD OF FOOD ORDERING SYSTEM



Requirement Analysis

Data Flow Diagram

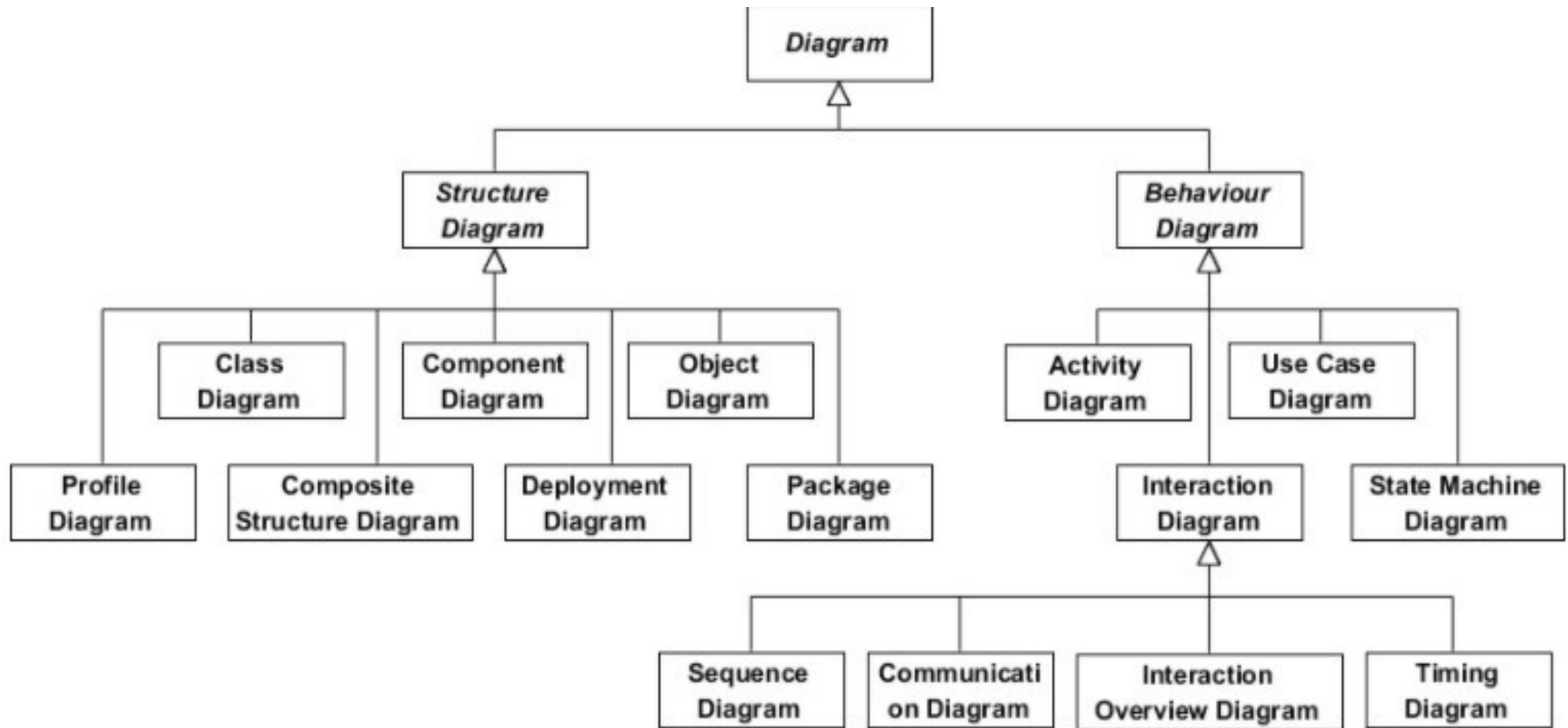
Level-1 DFD , showing decomposition of process 1.0



Unified Modeling Language

- UML is a standardized visual modeling language used to specify, visualize, construct, and document the artifacts of a software system.
- It is not a programming language, but a diagramming language that helps developers, analysts, and designers understand software structure and behavior.

Unified Modeling Language



Requirement Analysis

In requirement analysis, we can use the following diagrams.

- Use Case
- Activity
- Activity Variant (swim lane diagram)

Supporting Diagrams (Sometimes used)

- Sequence Diagram
- State Machine Diagram

Use Case Diagram

It describes the functional requirements of a system by showing how users interact with the system.

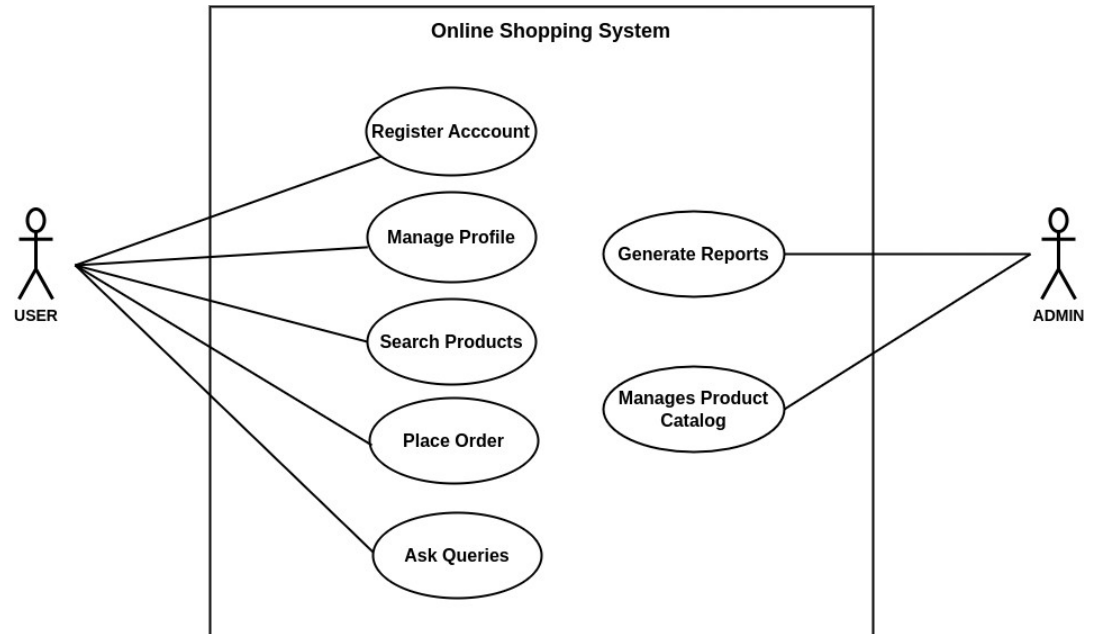
A Use Case diagram shows what a system should do from the user's perspective.

Main Elements

Actor (user or external system)

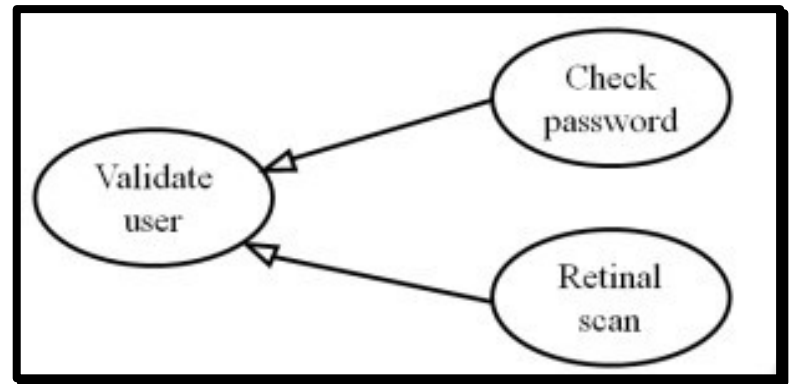
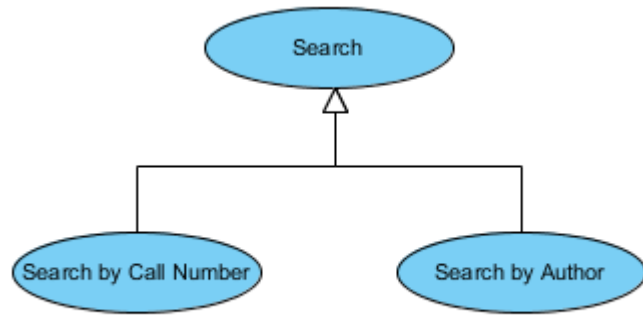
Use Case

System boundary



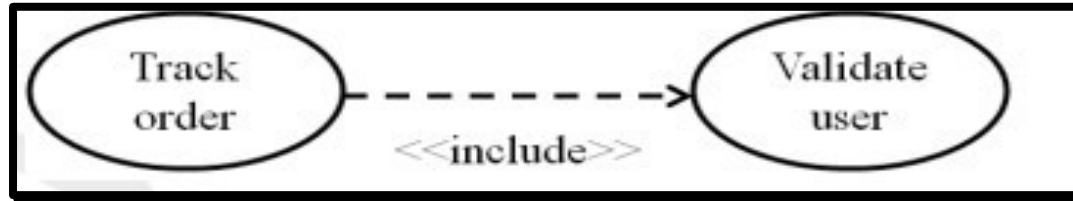
Use Case Diagram

Generalization and Specialization



Use Case Diagram

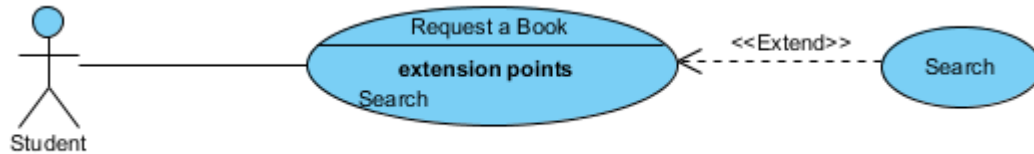
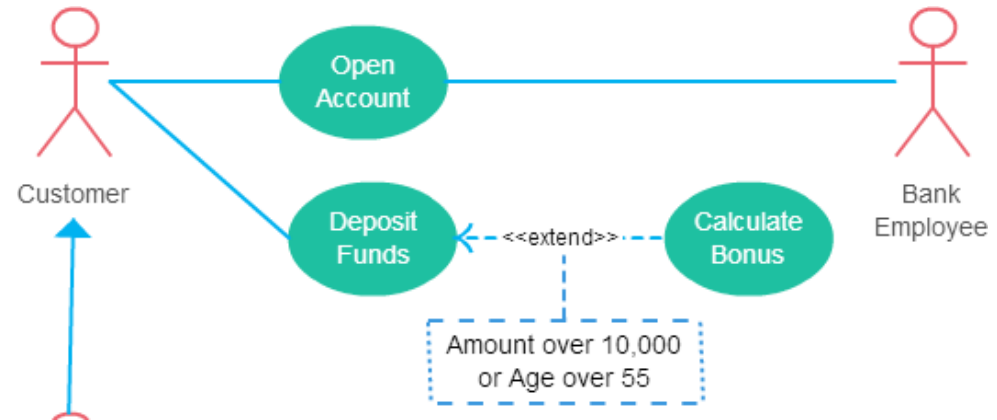
Include



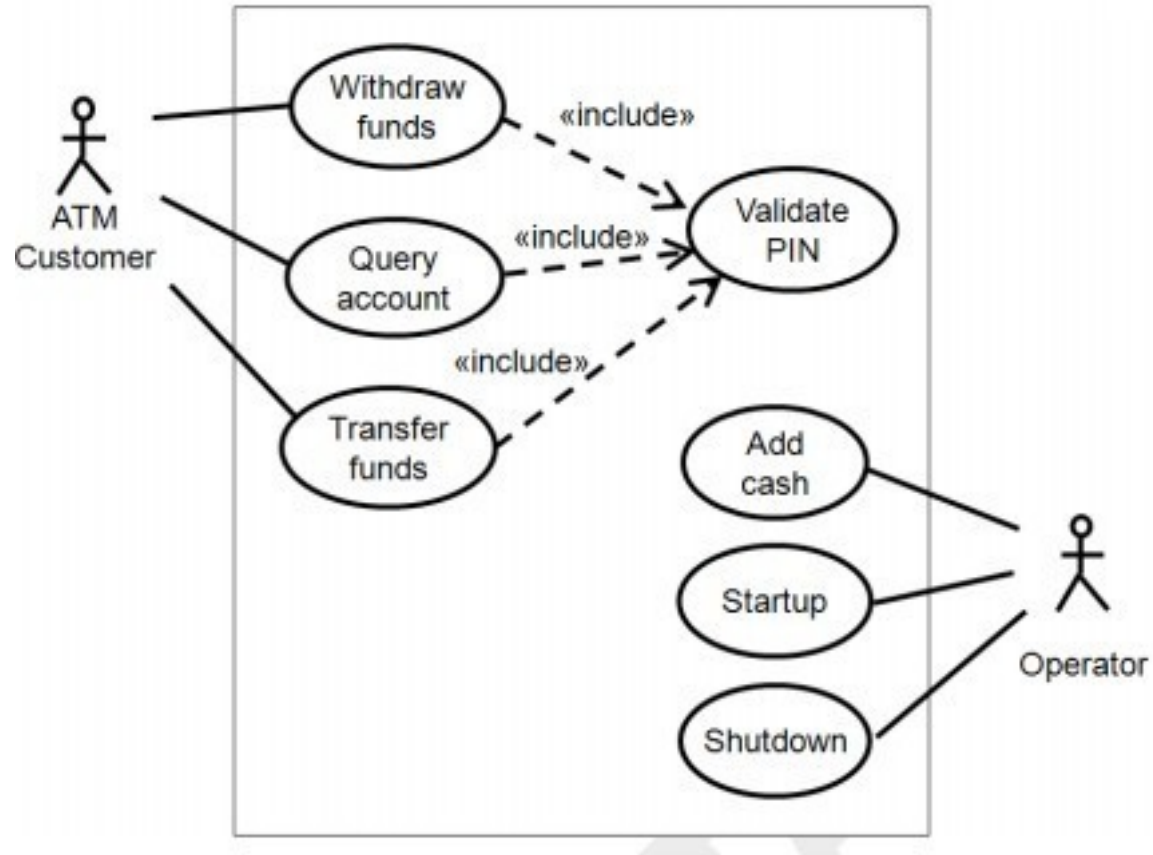
Use Case Diagram

Extend

Extend use case is optional and can be triggered conditionally.



Use Case Diagram of ATM



Use Case Template

Name	Validate PIN
Summary	System validates customer PIN
Dependency	none
Actors	ATM, Customer
Preconditions	ATM is idle, displaying a Welcome message.
Flow of Events	Activity Diagram
Alternatives	• If the system does not recognize the card, the card is ejected. • If the system determines that the card date has expired, the card is confiscated. • If the system determines that the card has been reported lost or stolen, the card is confiscated. • If the customer-entered PIN does not match the PIN number for this card, the system re-prompts for PIN. • If the customer enter the incorrect PIN three times, the system confiscates the card.
Post condition	Customer PIN has been validated.

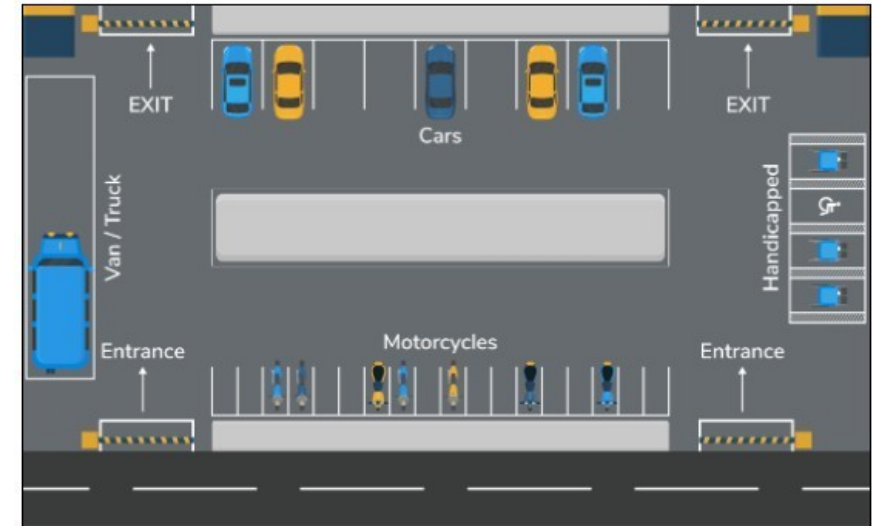
Case Study: Parking Lot

In a parking lot, there are a fixed number of parking spots available for different types of vehicles.

Each of these spots is charged according to the time the vehicle has been parked in the parking lot.

The parking time is tracked with a parking ticket issued to the vehicle at the entrance of the parking lot.

Once the vehicle is ready to exit, it can either pay at the automated exit panel or to the parking agent at the exit using a card or cash payment method.



Requirement Elicitation: Interview

An overview of some major expectations the interviewer will want an interviewee to discuss in more detail during the interview.

Payment:

How are customers able to pay at different exit points (i.e., either at the automated exit panel or to the parking agent) and by different methods (cash, credit, coupon)?

If there are multiple floors in the parking lot, how will the system keep track of the customer having already paid on a particular floor rather than at the exit?

Requirement Elicitation: Interview

Parking spot

What happens when a lot becomes full?

How can one keep track of the free parking spots on each floor if there are multiple floors in the parking lot?

If the parking spot of any vehicle type is booked, can a vehicle of another type park in the designated parking spot?

Requirement Elicitation: Interview

Pricing

How will pricing be handled? Should we accommodate having different rates for each hour? For example, customers will have to pay \$4\$4 for the first hour, \$3.5\$3.5 for the second and third hours, and \$2.5 for all the subsequent hours.

Will the pricing be the same for the different vehicle types?

Requirements Collection

R1: The parking lot should have the capacity to park 40,000 vehicles.

R2: The four different types of parking spots are handicapped, compact, large, and motorcycle.

R3: The parking lot should have multiple entrance and exit points.

R4: Four types of vehicles should be allowed to park in the parking lot, which are as follows:

- Car
- Truck
- Van
- Motorcycle

Requirements Collection

R5: The parking lot should have a display board that shows free parking spots for each parking spot type.

R6: The system should not allow more vehicles in the parking lot if the maximum capacity (40,000) is reached.

R7: If the parking lot is completely occupied, the system should show a message on the entrance and on the parking lot display board.

Requirements Collection

R8: Customers should be able to collect a parking ticket from the entrance and pay at the exit.

R9: The customer can pay for the ticket either with an automated exit panel or pay the parking agent at the exit.

R10: The payment should be calculated at an hourly rate.

R11: Payment can be made using either a credit/debit card or cash.

Actors

System

Our system is a "parking lot."

Primary actors

Customer: This actor can park the vehicle in the allocated parking space according to the vehicle type and pay for the parking before exit.

Parking agent: The parking agent will assist the customer and perform all the tasks that a customer can do, such as paying the parking ticket on behalf of the customer.

Actors

Secondary actors

Admin: This can add, remove, or update a spot, agent, entry/exit panels, and view/update accounts.

System: This is responsible for giving details of parking spot availability and assigning a parking spot to a vehicle.

Use Cases

Customer

Take ticket: To take a ticket at the entrance, that contains information regarding the vehicle and its entrance time

Scan ticket: To scan the ticket at the exit and get the parking fee

Pay ticket: To pay the parking fee at the exit panel via cash or a credit card

Cash: To pay the parking fee via cash

Credit card: To pay the parking fee via credit card

Park vehicle: To park the vehicle at the assigned destination

Use Cases

Parking agent

Update account: To update account details and payment information

Login/Logout: To log in/log out to/from the agent or admin account

View account: To view account details like payment status or unpaid amount

Take ticket: To take a ticket at the entrance, that contains information regarding the vehicle and its entrance time

Scan ticket: To scan the ticket at the exit and get the parking fee

Pay ticket: To pay the parking fee at the exit panel via cash or a credit card

Cash: To pay the parking fee via cash

Credit card: To pay the parking fee via credit card

Park vehicle: To park the vehicle at the assigned destination

Use Cases

Admin

Add spot: To add a parking spot

Add agent: To add a new agent

Add/modify rate: To add/modify hourly rate

Add entry/exit panel: To add and update exit/entry panel at each entry/exit

Update account: To update account details and payment information.

Login/Logout: To login/logout to/from agent or admin account

View account: To view account details like payment status or unpaid amount

Use Cases

System

Assigning parking spots to vehicles: To check the vehicle type and associate a free spot according to it

Remove spot: To remove a parking spot if it is not available for parking

Show full: To display the status of the parking lot as full

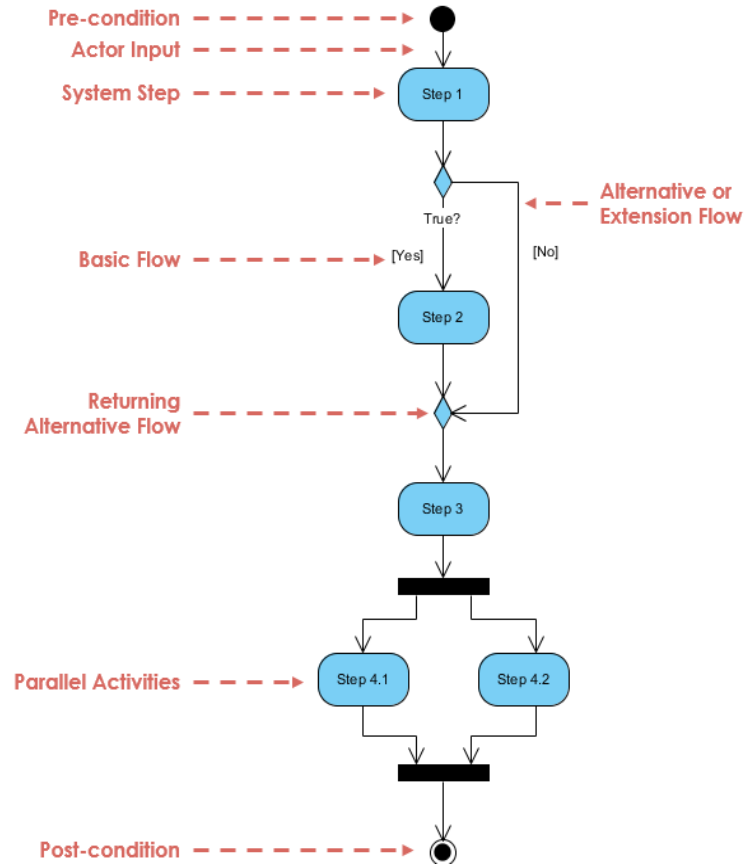
Show available: To show the details of available parking spots

Behavioral Modeling

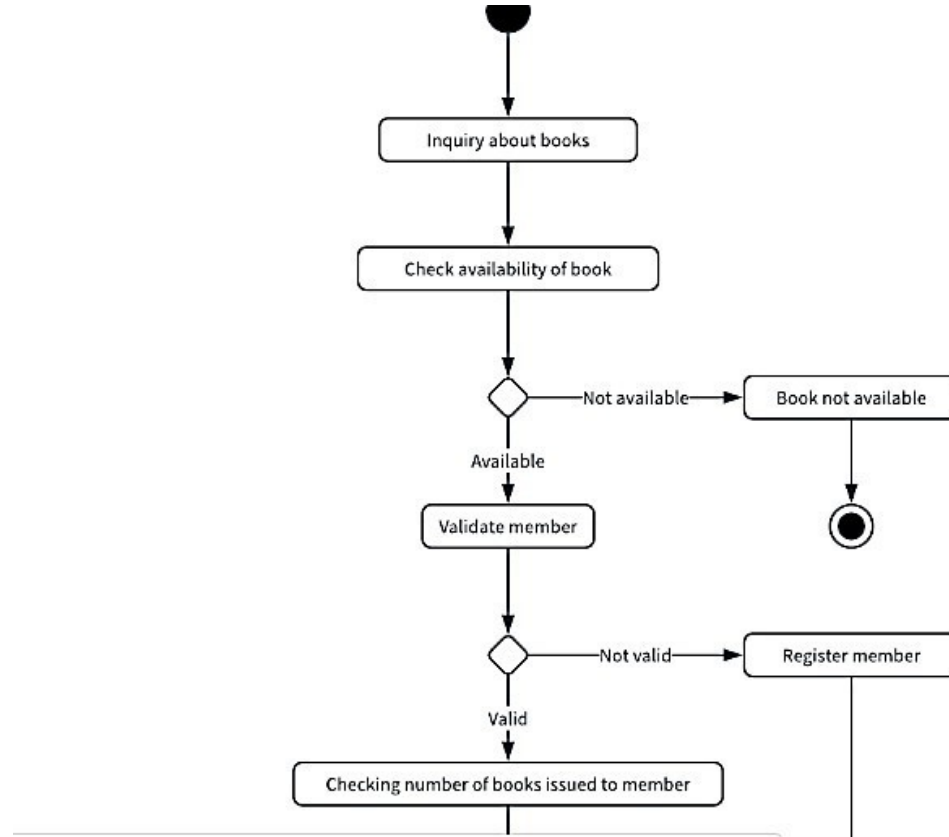
An activity diagram can be used to capture the activities and actions performed in a use case.

It expresses the dynamic aspect of the system.

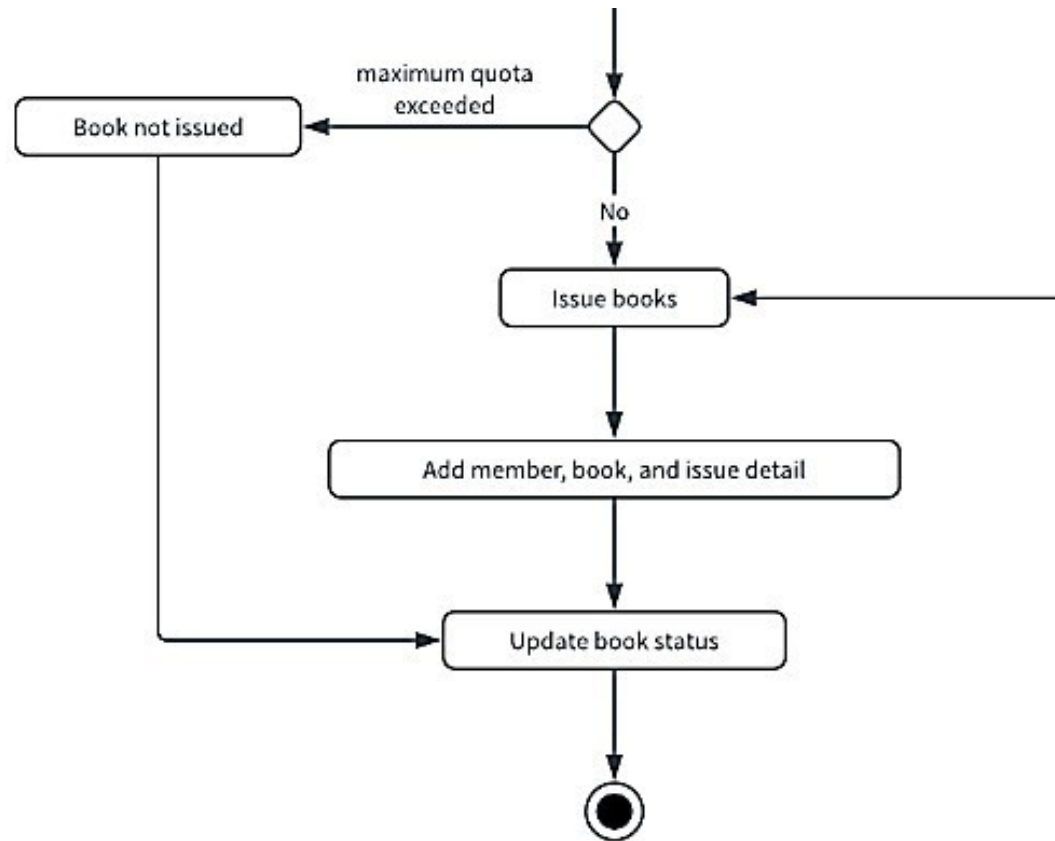
Behavioral Modeling (Activity Diagram)



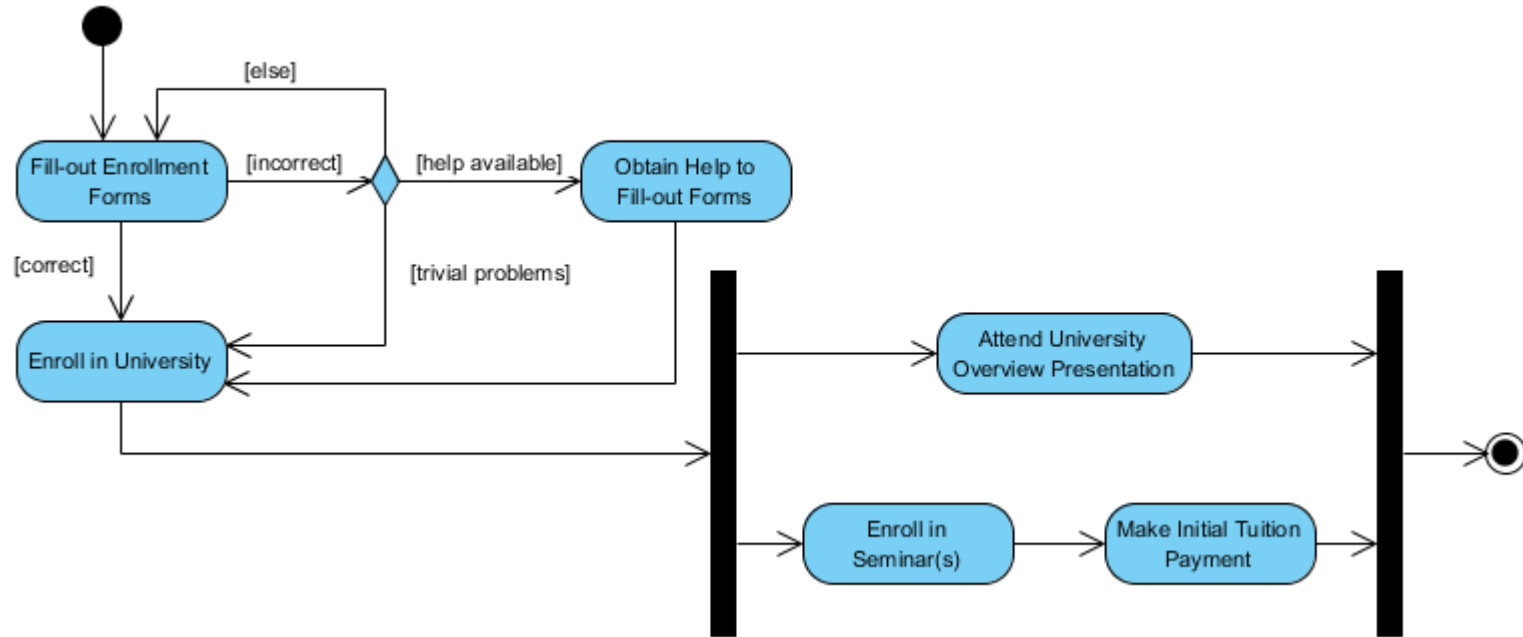
Behavioral Modeling (Activity Diagram)



Behavioral Modeling (Activity Diagram)



Behavioral Modeling (Activity Diagram)



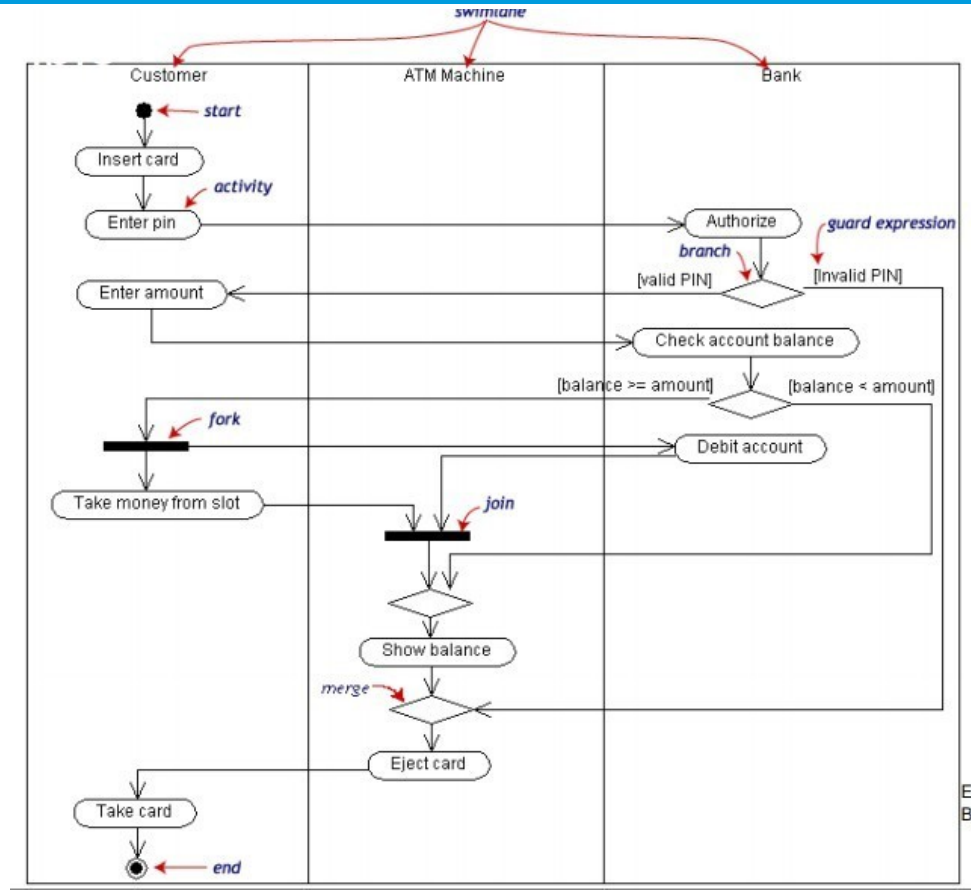
Behavioral Modeling (Swimlane Diagram)

Mapping Who Does What to Whom

You are assigning a responsibility to an actor.

Note, we did not say to an object - to an actor.

Behavioral Modeling (Swimlane Diagram)



Behavioral Modeling (State Machine Diagram)

A state machine diagram models dynamic behavior.

It specifies the sequence of states in which an object can exist:

State machine diagrams are usually applied to objects but can be applied to any element that has behavior to other entities such as: actors, use cases, subsystems systems etc.

Elements of State Machine Diagram

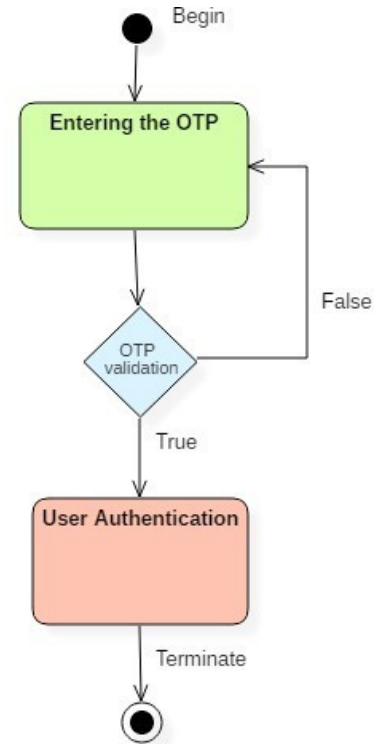
States

Events

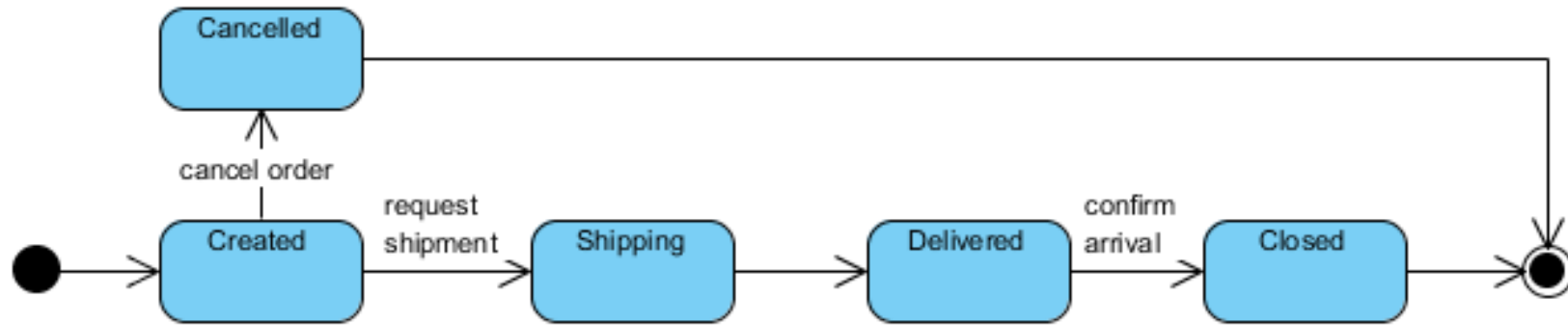
Transition

An object has state, behavior, and a unique identity.

An object goes through various states during its lifespan.



State Machine Diagram



Requirements Validation

Requirement validation is the process of checking that documented requirements are correct, complete, consistent, feasible, and truly reflect stakeholder needs.

Requirements Validation

A requirement is valid if it is

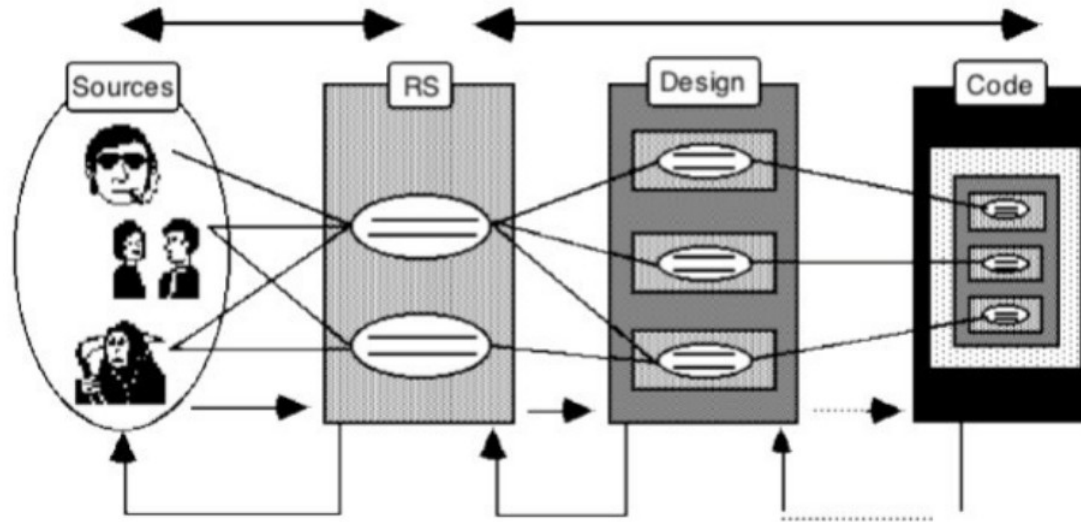
- Correct (Matches real user needs)
- Complete (Nothing missing)
- Consistent (No contradictions)
- Unambiguous (Only one meaning)
- Feasible (Can be implemented)
- Verifiable (Can be tested)



Reviews / Inspections

Requirement Traceability Matrix

A Requirements Traceability Matrix (RTM) is a document that links requirements with their corresponding design, implementation, and test cases to ensure complete coverage and traceability.



Requirement Traceability Matrix

Requirements Traceability Matrix									
Project Name	<Project Name here>		Created On	3-Oct-11	Reviewed On	3-Oct-11			
Release No	<Project Release>		Created By	<Creator's Name>	Reviewed By	<Reviewer's Name>			
Version	<Doc version>								
ID	Requirement ID	Requirement Description	Status	Design Document	Code Module	TestCase ID	Test Case Name	User Manual	Tested On/ Verification
001	UC 1.1	Testing Requirement Description here. It should not be more than 2-3 lines	Status	DM-001	CM-001	TC-001		Section 4.5	Pending
002	UC 1.1	Testing Requirement Description here. It should not be more than 2-3 lines	Approved	DM-002	CM-002	TC-002 TC-003		Section 4.6	Verified
004	UC 1.3	Testing Requirement Description here. It should not be more than 2-3 lines	Status	DM-003	CM-004	TC-004 TC-006 TC-007 TC-008		Section 5.7	Verified
004	UC 1.3	Testing Requirement Description here. It should not be more than 2-3 lines	Approved	DM-003	CM-004			Section 4.6	In-progress
005	UC 1.4	Testing Requirement Description here. It should not be more than 2-3 lines	Approved	DM-005	CM-005			Section 7.8	In-Verified
006	UC 1.5	Testing Requirement Description here. It should not be more than 2-3 lines	TBD	DM-006	CM-006			Section 6.10	Verified
007	UC 1.6	Testing Requirement Description here. It should not be more than 2-3 lines	Approved					Section 4.11	Not Verified
008	UC 1.7	Testing Requirement Description here. It should not be more than 2-3 lines	TBD					Section 4.12	Pending
008	UC 1.9	Testing Requirement Description here. It should not be more than 2-3 lines	TBD					Section 4.13	Not Verified
010	UC 1.9	Testing Requirement Description here. It should not be more than 2-3 lines	Approved					Section 4.14	Pending

Requirement Document Template

1: Introduction

1.1 Purpose

Why this document exists.

Example:

This document describes the functional and non-functional requirements of the Online LMS.

1.2 Scope

What the system will and will not do.

Example:

The system will allow students to enroll in courses, watch lectures, and take quizzes.

1.3 Definitions & Acronyms

Explain technical terms.

Requirement Document Template

2: Overall Description

2.1 Product Perspective

Is it new or part of existing system? Product details

2.2 Product Functions

A high-level summary of the main features

2.3 User Classes

Who will use it?

Example: Admin, Instructor, Student

2.4 Operating Environment

Web / Mobile / Cloud

2.5 Design & Implementation Constraints

Any specific languages, hardware limits, etc

Example:

Use java

Must use MySQL

Requirement Document Template

3: Requirements

3.1 Business Requirements

3.2 User Requirements

3.3 Functional Requirements (covers technical details)

3.4 Non Functional Requirements

3.5 Data Requirements

3.6 External interface Requirements

- User Interfaces: Layout standards, buttons, and navigation etc.
- Hardware Interfaces: Does it need to talk to a printer, a sensor, or a specific server?
- Software Interfaces: Connections to databases, or third-party APIs.
- Communications Interfaces: Protocols like HTTP, FTP, or Bluetooth.

Requirement Document Template

4: System Models

- DFD's
- Use Cases
- Activity / Swimlane
- State Machine

5: Requirement Traceability Matrix

Have A Good Day!